

EVENT:

DECENTRALIZED EVENT TICKETING AND ANTI-SCALPING PROTOCOL

SECTION:

ISCS 30.66 - MT

HOLDER:

ANG, Christian
KATIGBAK, Kryzelle
GARROVILLO, Javi
ONG, Hans
PINTOR, Ryan



WHAT'S THE PROBLEM?

- **Scalping bots bulk-buy tickets**, creating artificial scarcity and inflated prices
- Fraudulent resales (faked QR codes) with **no way to verify authenticity**
- Buyers are either **priced out** or **scammed** on secondary markets



WHY BLOCKCHAIN & SMART CONTRACT?

- **Blockchain provides an immutable, tamper-proof ledger**
 - Ticket ownership is cryptographically verified, so fakes are impossible
- **Smart contracts enforce rules automatically**
 - No middleman needed
- **Price ceilings are coded into the contract via require statements**
 - Scalpers physically cannot list above the cap
- All transactions are transparent and auditable on-chain



SYSTEM REQUIREMENTS

- Mint event tickets as **individual smart contracts**
- **Support batch minting**
 - Up to 50 tickets per transaction
- **Allow primary sales**
 - Organizer lists and Buyer purchases
- **Enforce a resale price ceiling**
 - 110% of Original Price
- **Support free transfers**
- Handle event cancellation with **automatic refunds**



SUPPORTED TRANSACTIONS

- **createTicket / createTickets**
 - Deploy ticket contracts
- **listForSale / unlistForSale**
 - Manage listings
- **buyTicket**
 - Purchase with anti-scalping enforcement
- **transferTicket**
 - Gift to another wallet
- **cancelEvent**
 - Cancel + Refund all holders



ARCHITECTURE OVERVIEW

- **Factory pattern**
 - EventOrganizer (factory) deploys individual Ticket contracts
- **EventOrganizer** tracks all tickets, handles batch operations, and cancellations
- Each **Ticket contract** manages its own ownership, status, pricing, and transfers
- **Ticket lifecycle**
 - Available → ForSale → Sold (can re-list) → Cancelled



SMART CONTRACT

```
279 // ----- TICKET CREATION -----
280
281 /// @notice Create a single ticket
282 /// @param _eventName Name of the event
283 /// @param _price Base price in Wei
284 function createTicket(string memory _eventName, uint256 _price) external onlyOrganizer { infinite gas
285     require(!eventCancelled, "Event has been cancelled");
286
287     // Deploy a new Ticket child contract and store its address
288     Ticket newTicket = new Ticket(_eventName, _price, address(this));
289     allTickets.push(newTicket);
290     currentEventName = _eventName;
291
292     emit TicketCreated(address(newTicket), _eventName, _price);
293 }
```



SMART CONTRACT

```
295  /// @notice Create multiple tickets in one transaction (batch minting)
296  /// @param _eventName Name of the event
297  /// @param _price Base price in Wei (same for all tickets in this batch)
298  /// @param _count How many tickets to create (1 to 50)
299  /// @dev The 50-ticket cap prevents the transaction from running out of gas.
300  ///      Each ticket deployment costs ~200k-300k gas, so 50 tickets = ~15M gas,
301  ///      which fits within typical block gas limits.
302  function createTickets(string memory _eventName, uint256 _price, uint256 _count) external onlyOrganizer {  infinite gas
303      require(!eventCancelled, "Event has been cancelled");
304      require(_count > 0 && _count <= 50, "Batch size must be 1-50");
305
306      for (uint256 i = 0; i < _count; i++) {
307          // Deploy a new Ticket contract and add it to the master list
308          Ticket newTicket = new Ticket(_eventName, _price, address(this));
309          allTickets.push(newTicket);
310          emit TicketCreated(address(newTicket), _eventName, _price);
311      }
312      currentEventName = _eventName;
313  }
```


SMART CONTRACT

```
316 // ----- EVENT MANAGEMENT -----
317
318 /// @notice Cancel the entire event and refund all ticket holders
319 /// @dev This function:
320 ///     1. Sets eventCancelled to true (blocks future ticket creation)
321 ///     2. Loops through every ticket and marks it as Cancelled
322 ///     3. Refunds originalPrice to every current ticket holder
323 ///     The factory must have enough ETH balance to cover refunds.
324 ///     It accumulates ETH from primary sales automatically.
325 function cancelEvent() external onlyOrganizer {  infinite gas
326     require(!eventCancelled, "Event already cancelled");
327     eventCancelled = true;
328
329     uint256 count = allTickets.length;
330     for (uint256 i = 0; i < count; i++) {
331         Ticket ticket = allTickets[i];
332         ticket.cancelTicket(); // Mark the ticket as cancelled
333         ticket.refundBuyer(); // Send originalPrice back to the holder
334     }
335
336     emit EventCancelled(currentEventName, count);
337 }
```



SMART CONTRACT

```
168    /// @notice Gift/transfer this ticket to someone else for free
169    /// @param _newOwner The wallet address of the recipient
170    /// @dev This covers the real-world case of giving your ticket to a friend.
171    ///       The ticket must NOT be currently listed for sale or cancelled.
172    function transferTicket(address _newOwner) external onlyOwner { 36025 gas
173        require(status != Status.Cancelled, "Cannot transfer a cancelled ticket");
174        require(status != Status.ForSale, "Unlist the ticket before transferring");
175        require(_newOwner != address(0), "Cannot transfer to zero address");
176        require(_newOwner != msg.sender, "Cannot transfer to yourself");
177
178        address previousOwner = currentOwner;
179        currentOwner = _newOwner;
180
181        // Price is 0 to distinguish gifts from sales in event logs
182        emit TicketSold(address(this), previousOwner, _newOwner, 0);
183    }
```



CONTRACT DEMO



METAMASK ACCOUNT ADDRESS

Name	Address
Organizer	0x8928d7d1a6d0cb5ea541af5253fd7d73a6cd080c
First Buyer	0x84d4dbee1f4ee6185edd5d9556c77386ed3b28a1
Second Buyer	0x77d751260d4b78e344974c7de2a34b397469f669



CONTRACT DEPLOYMENTS

Deployed Contracts 3

+ Add Contract 

Interact with a deployed contract

 EventOrganizer
0xC56...9A14D 

Sepolia
3min ago 

 Ticket
0x0bd...Db4da 

Sepolia
23min ago 

 Ticket
0x3f4...9D348 

Sepolia
2min ago 

BATCH MINTING

● createTickets string, uint256, uint256

UAAP Finals 2026

100

2

Value

0

wei

Gas limit

auto

0

Transact

Balance

0 ETH

● allTickets uint256

0

0:address: 0x0bd31ac608A8A687300f009
ABe1F8F328E9Db4da

Call

● allTickets uint256

1

0:address: 0x3f4842d38BA50bE8e66fC410
7e21F93A9769D348

Call

LISTING TICKET FOR SALE

● listTicketForSale uint256, uint256

0

1000

Value

0

wei

Gas limit

auto

0

Transact

Balance

0 ETH

BUYING TICKET

First Buyer 

0x84d...b28a1 

0.008 ETH

● buyTicket

Value  wei ▼

Gas limit auto

Transact

Balance 0 ETH

First Buyer 

0x84d...b28a1 



● currentOwner

0:address: 0x84d4DbEE1F4Ee6185edd5D9
556C77386ED3B28A1

Call

Balance 0 ETH

FREE TRANSFER FEATURE

● **transferTicketTo** uint256, address

1

0x77d751260d4b78e344974c7de2a34b397469f669

Value

0

wei

Gas limit

auto

0

Transact

Balance

0 ETH

Ticket

0x3f4...9D348

Sepolia

26min ago

Functions

Select a function to interact with...

Low level interaction

● **currentOwner**

0:address: 0x77d751260D4B78E344974C7
DE2a34b397469f669

Call

ANTI-SCALPING FEATUTRE

●

listForSale uint256

2000

Value

0

wei

Gas limit

auto

0

Transact

Balance

0 ETH

[illegible]

VALID RESALE

First Buyer 

0x84d...b28a1 

0.018 ETH

● listForSale

uint256

1100

Value

0

 wei ▼

Gas limit

auto

0 

Transact

Balance

0 ETH

TRANSACTION HASHES

Action	Tx Hash
Deploy EventOrganizer	0xeb50de9d06a6e60c5527ebd4529c3b493a7eb79a4c1d188422582bb5acc86ad5
Batch mint 2 tickets	0xca33b49117d2d1a46b377a246add30358278ce4620a4bb3d9ed5fb3069f68e49
List Ticket #0 at 1000 Wei	0x760662375332bf96e86abc3fbf201efccf7a9b7b2501021a5986d351403b3a58
Buy Ticket #0 (First Buyer)	0x1f2b442af40a12208c535e91593b063618901a7da43a8f76875e1160fb040fdc
Scalp attempt at 2000 Wei	N/A
Valid resale at 1100 Wei	0xf37119a79d7ab4c490d84393ddb03f6d2c89b53566c6e69377e45cd611d8c6eb
Gift Ticket #1 to Second Buyer	0x119e49aae3eea6b39e93378d9d66adcb80df71f80e6da81a3f0bd1b648f8cdab



CONCLUSION

- **Scalping has been very prevalent** as it has been an opportunity to profit off people
 - Smart contracts can be the **future for any ticketing mechanisms**
- This can be **combined with a proper web frontend for a seamless user experience**



**THANK
YOU**

